

The Reproducibility Crisis of LLM Benchmarks in Software Engineering

Gustavo A. Oliva, *Queen's University, Kingston, ON, K7L 3N6, Canada*

Dayi Lin, *Queen's University, Kingston, ON, K7L 3N6, Canada*

Ahmed E. Hassan, *Queen's University, Kingston, ON, K7L 3N6, Canada*

Abstract—The integration of Large Language Models (LLMs) into Software Engineering (SE) has spurred the creation of benchmarks like SWE-Bench and LiveCodeBench to measure their capabilities. However, current reporting practices fail, causing a reproducibility crisis driven by two fundamental problems: inherent non-determinism and parameter sensitivity. Even with identical configurations, LLM benchmark results vary significantly across runs due to the stochastic nature of model inference. Additionally, benchmarks depend on numerous parameters (inference settings, scaffold versions, environment configurations), often unreported. This paper presents a two-tiered solution: immediate recommendations that practitioners can adopt today (standardized configuration reporting and mandatory distribution reporting instead of single values), and long-term recommendations requiring community coordination (crowdsourced statistical benchmarking and standardized benchmark engineering). Drawing from our industrial experience, we argue that both tiers are essential for establishing robust, trustworthy baselines enabling fair comparison and progress in LLM evaluation for software engineering.

The rapid advancement of Large Language Models (LLMs) has created a paradigm shift in software engineering (SE), with models now capable of generating code, fixing bugs, and even designing entire software components. To measure their progress, the community has developed benchmarks like SWE-Bench, LiveCodeBench, and the Aider Polyglot benchmark. However, the current practices for reporting results are failing, leading to a crisis in reproducibility, transparency, and trustworthiness.

This crisis affects multiple stakeholders across the LLM ecosystem. Model providers struggle to report results responsibly while demonstrating their models' capabilities. Practitioners evaluating models for production deployment lack the transparency needed to make informed decisions and validate vendor claims. Teams fine-tuning models cannot establish trustworthy baselines to measure their improvements. Researchers publishing LLM evaluations need practices that make their results verifiable and comparable. Each

of these audiences faces the same fundamental challenge: current benchmarking practices provide neither the reproducibility nor the statistical rigor necessary for scientific progress or practical decision-making.

In this paper, we distill our team's practical experience into actionable recommendations addressing two fundamental problems that plague current benchmarking practices. First, inherent non-determinism means that even with identical configurations, benchmark results vary significantly across runs due to the stochastic nature of LLM inference. Second, parameter sensitivity means that small changes in configuration (often undocumented) can drastically alter outcomes. We demonstrate both problems with empirical data from our own evaluation efforts. We then present a two-tiered solution framework: immediate recommendations that individual researchers and companies can adopt today (comprehensive configuration reporting and mandatory distribution reporting instead of single values), and long-term recommendations that require community coordination (crowdsourced statistical benchmarking and standardized benchmark engineering). Our goal is to move the community towards

a more robust, reliable, and scientific foundation for evaluating LLMs in software engineering.

MOTIVATING EXAMPLE

To illustrate the severity of the reproducibility crisis, we present empirical evidence from our own evaluation efforts demonstrating two distinct but equally problematic issues: inherent non-determinism and parameter sensitivity. Together, these problems make current single-point benchmark reporting fundamentally unreliable.

Problem 1: Inherent Non-Determinism

Even when all configuration parameters remain constant, LLM benchmark results exhibit significant variation across runs. To demonstrate this, we executed the Aider Polyglot benchmark ten times on the same model using identical configurations (see Figure 1). The pass rates varied substantially across runs, with differences large enough to change the ranking of models on public leaderboards. In the current era of "benchmaxing," where every decimal point is scrutinized and minor improvements are celebrated, this level of variation is deeply problematic.

A common misconception is that setting the temperature parameter to zero ensures deterministic output. However, this approach fails for two reasons. First, greedy decoding (temperature equals zero) does not guarantee determinism across all LLM implementations and hardware configurations. Second, and more critically, several reasoning models fail or produce degraded results when temperature is set to zero.¹ This makes temperature-based determinism impractical for comprehensive benchmarking.

The implication is clear: single-point benchmark results are fundamentally unreliable. A model that scores 65% on one run might score 62% or 68% on another, purely due to the stochastic nature of inference. Without understanding this distribution, we cannot meaningfully compare models or track progress.

Problem 2: Parameter Sensitivity

Beyond inherent non-determinism, benchmark results are highly sensitive to configuration parameters, many of which are poorly documented or outright discouraged by scaffold documentation. We executed SWE-Bench using OpenHands with its default parameters, achieving a specific pass rate. We then modified a single OpenHands parameter, one that the documentation actually discourages enabling, and observed a dramatic improvement in performance. This parameter

was not highlighted in published results, yet it fundamentally changed the outcome.

This example illustrates a broader problem: the configuration space for LLM benchmarks is vast and poorly understood. Parameters span multiple layers (model inference settings, scaffold behavior, execution environment), and their interactions are complex. Without comprehensive reporting of all parameters, published results become irreproducible and comparisons become meaningless. A model appearing superior might simply benefit from undisclosed parameter tuning rather than genuine capability improvements.

RECOMMENDATIONS

We organize our recommendations into two tiers based on their implementation timeline and coordination requirements. Immediate recommendations can be adopted today by individual researchers and companies. Long-term recommendations require community-wide coordination but offer transformative improvements to the benchmarking ecosystem.

IMMEDIATE RECOMMENDATIONS

These recommendations address the most pressing issues and can be implemented immediately without requiring community-wide coordination.

Comprehensive and Standardized Configuration Reporting

A fundamental barrier to reproducibility is the ad-hoc and incomplete manner in which benchmark configurations are currently reported. To address this, we propose a standardized, multi-part reporting structure that covers all layers of the evaluation stack. This structure should be captured in a `benchmark_spec.yaml` file to ensure clarity and consistency.

Model Specification Trust in benchmark results begins with knowing precisely which model was evaluated. Currently, submissions often only specify a model's name (e.g., "Llama-3-70B"). This is insufficient, as fine-tuned variants can have vastly different capabilities. We propose that all submissions must specify the exact base model and include a cryptographic hash (e.g., SHA-256) of the model weights for any open-source model. This provides an unambiguous, verifiable fingerprint of the exact artifact used.

Inference Engine Parameters The software used to serve the model can significantly impact performance.

Aider Polyglot Dashboard

Showing 10 experiments

Summary (Figure) Summary (Baseline view supported) Per Language Detailed View

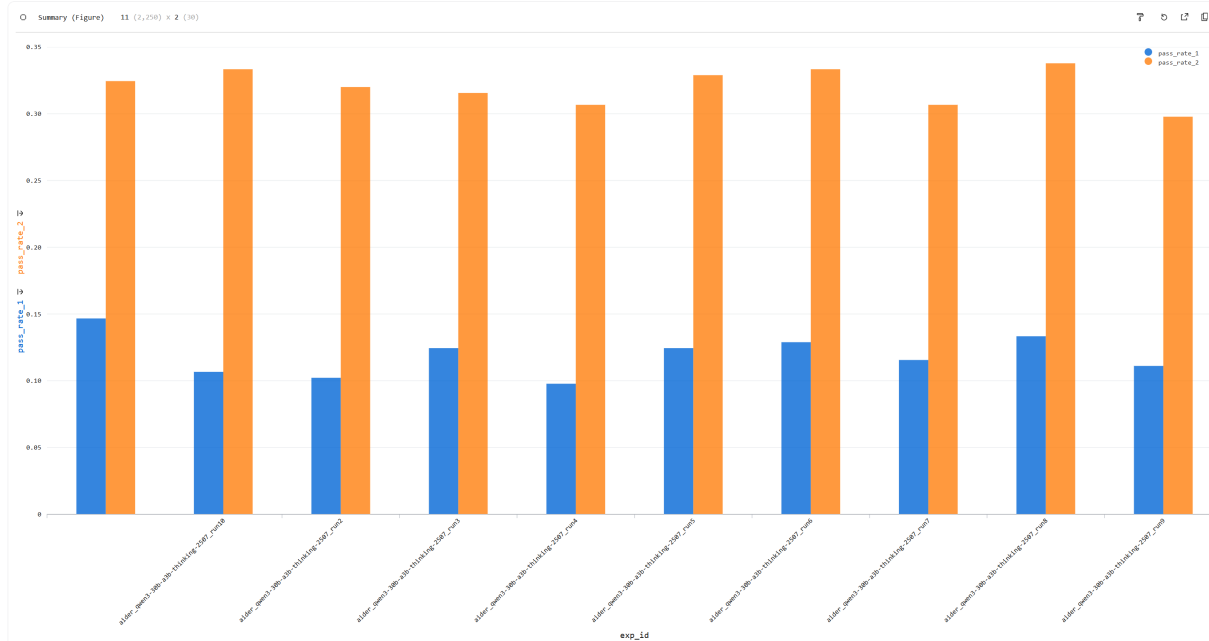


FIGURE 1. Pass rates across ten independent runs of the Aider Polyglot benchmark using identical configurations. The variation in results demonstrates inherent non-determinism in LLM evaluation, with differences large enough to affect model rankings on public leaderboards.

Parameters related to the deployment and optimization of the inference engine, such as the configuration of vLLM or TensorRT-LLM, are often overlooked. This includes settings like the level of quantization, data-parallel size, and other engine-specific tunings. These parameters must be explicitly reported to understand the full context of the execution environment.

Inference Parameters The parameters used at the time of generation directly control the model's output behavior. While some benchmarks report basic settings like temperature, a complete picture is necessary. We recommend the mandatory reporting of all inference parameters, including but not limited to temperature, top-p, top-k, repetition penalty, and maximum token count.

Scaffold Parameters The "scaffold" or "harness" that orchestrates the interaction between the model and the benchmark tasks is a major source of variability. Frameworks like OpenHands for SWE-Bench have their own configuration parameters that can dramatically influence outcomes. These settings, which control aspects like the agent's strategy or the number of

interaction turns, must be treated as first-class citizens in the configuration report.

Execution Environment When a benchmark is not executed within a provided container, the host environment becomes a significant source of variability. We recommend that all non-containerized benchmark runs must report a complete snapshot of the execution environment. This includes the operating system, Python version, and the specific version of critical hardware drivers like CUDA. Furthermore, a full list of pinned library dependencies, such as the output of `pip freeze`, must be included to prevent discrepancies arising from package updates.

Report Distributions, Not Single Values

As demonstrated in our motivating example, inherent non-determinism means that single-point benchmark results are fundamentally unreliable. We recommend that all benchmark publications, whether in research papers or on model leaderboards, must report results from multiple runs rather than a single execution. Specifically, we propose the following requirements.

Minimum Run Count

Benchmark results should be reported based on a minimum of three to five independent runs using identical configurations. While this increases computational cost, it is the only way to understand the true performance distribution and account for stochastic variation.

Statistical Measures

Instead of reporting a single number, publications must include the mean, median, and standard deviation of the pass rate (or other primary metric) across all runs. This provides a complete picture of both central tendency and variability. For instance, a model with a mean pass rate of 65% and a standard deviation of 3% is fundamentally different from one with the same mean but a standard deviation of 8%.

Critique of Single-Number Leaderboards

Public leaderboards that display only a single score per model are misleading and should be considered unreliable. Without understanding the variance, we cannot know whether a one-point difference between two models is meaningful or simply noise. Leaderboard maintainers should require submitters to provide multiple run data and display confidence intervals or ranges alongside point estimates.

This recommendation serves as a practical mitigation strategy that can be implemented immediately while the community works towards the long-term solution of crowdsourced statistical benchmarking. Even in the absence of a centralized repository, individual researchers and companies can adopt this practice to provide more trustworthy results.

LONG-TERM RECOMMENDATIONS

While the immediate recommendations can be adopted today, the following recommendations require community-wide coordination and infrastructure development. However, they offer transformative improvements to the benchmarking ecosystem.

Shift to Crowdsourced, Statistical Benchmarking

The current model of relying on a few single-point results published in papers or by model vendors is fragile and fails to address both problems identified in our motivating example. We advocate for a paradigm shift towards a crowdsourced, statistical model inspired by mature fields like hardware benchmarking. We envision a central, public repository where the community can submit benchmark results, each accompanied by its detailed configuration file (as per our immediate

recommendation on configuration reporting). Critically, each submission would include multiple runs, providing a complete distribution rather than a single point.

This approach directly addresses inherent non-determinism by making variance visible and quantifiable. Instead of asking "what score did Model X achieve?", we would ask "what is Model X's performance distribution?" With sufficient community submissions, we could establish robust statistical measures (median, interquartile range, outlier detection) that provide a much more reliable signal of true model capabilities.

Simultaneously, this system addresses parameter sensitivity by exposing the impact of different configurations. When the repository contains multiple submissions for the same model with different parameter sets, we can empirically observe how sensitive performance is to specific settings. This transparency would discourage cherry-picked "hero runs" and reveal whether reported improvements stem from genuine model capabilities or merely undisclosed parameter tuning.

Such a system would serve as a powerful reality check for industrial teams. When evaluating whether to adopt a new model, we could compare our internal benchmark results against the community distribution to understand whether our outcomes are typical or anomalous. This would dramatically reduce wasted effort chasing reproducibility issues.

Standardized Benchmark Engineering

Executing LLM benchmarks is a time-consuming and computationally expensive endeavor. From both industrial and academic perspectives, integrating and running multiple benchmarks compounds this challenge significantly. Each benchmark is often a standalone, idiosyncratic system with its own unique setup and non-standard parameters, making scaled evaluation akin to a complex legacy system integration project. This lack of standardization leads to wasted resources and duplicated effort across organizations. We issue a call for the community to treat benchmarks as robust, interoperable software tools with standardized interfaces and practices that directly support reproducibility.

First, benchmarks should support a common, well-documented set of inference parameters and automatically generate the comprehensive configuration reports called for in our immediate recommendations. Rather than requiring users to manually assemble configuration details from multiple sources, the benchmark should output a complete `benchmark_spec.yaml` file capturing all relevant settings.

Second, benchmarks should be distributed as containerized applications (e.g., using Docker) to reduce execution environment variability. This mitigates the challenges discussed in our Execution Environment recommendation, where differences in operating systems, library versions, and hardware drivers can affect results. Containerization would also facilitate integration with the crowdsourced benchmarking infrastructure proposed earlier, enabling better tooling for analyzing configuration sensitivity and aggregating results across the community.

CONCLUSION

The software engineering community has built remarkable benchmarks that have fundamentally shaped our understanding of LLMs in software engineering. Now we must fix how we use them. Three insights should guide us forward:

- **Single-point benchmark results are unreliable.** The inherent non-determinism of LLM inference means that reported scores need statistical context. While multiple runs are computationally expensive, model providers publishing official performance results must show distributions (e.g., boxplots) rather than single numbers.
- **Configuration transparency is non-negotiable.** Every benchmark result must include a complete, machine-readable specification of all parameters across the evaluation stack. Without this, reproducibility remains impossible.
- **We need crowdsourced statistical benchmarking infrastructure.** The current model of isolated, vendor-reported scores limits our ability to establish trustworthy baselines. Community-driven repositories that expose performance distributions and configuration sensitivities would transform how we evaluate and compare models.

The immediate recommendations can be adopted today by individual practitioners and model providers. The long-term recommendations require coordination, but the path is clear. The transformative potential of LLMs in software engineering deserves an evaluation foundation built on transparency, statistical rigor, and community collaboration.

SUPPLEMENTARY MATERIAL

To practice what we preach, we provide complete configuration specifications for all benchmark executions presented in our motivating examples. These include comprehensive `benchmark_spec.yaml` files

covering model specifications, inference engine parameters, inference parameters, scaffold configurations, and execution environment details. Additionally, we provide a JSON schema defining the structure and required fields for benchmark configuration reporting. We emphasize that this schema is not final but rather a starting point for the community to improve upon through feedback and contributions. All materials are available in our GitHub repository: <https://github.com/placeholder/llm-benchmark-reproducibility>.

ACKNOWLEDGMENTS

The author(s) would like to thank A, B, and C. This work was supported by XYZ under Grant ###.

REFERENCES

1. C. Pipis, S. Garg, V. Kontonis, V. Shrivastava, A. Krishnamurthy, and D. Papailiopoulos, "Wait, wait, wait... why do reasoning models loop?" 2025. [Online]. Available: <https://arxiv.org/abs/2512.12895>

Example is a researcher at the ABC Corporation, Böblingen, Germany. Her current research interests include a, b, and c. Author received her Ph.D. degree in physics from University. She is a Fellow of the IEEE Computer Society. Contact her at sbauthor@abc.com.

Gustavo A. Oliva is ...

Dayi Lin is

Ahmed E. Hassan is